

NobleProg

The World's Local Training Provider



Sesión 1

Fundamentos y la justificación de migrar

Migración de Monolitos a Microservicios

Carlos Georg Lübbe · 3 h

Objetivo de hoy

Establecer un **vocabulario común** y un encuadre **honesto** de cuándo —y cuándo no— migrar.

1. Monolitos vs. microservicios: qué son de verdad
2. Más allá de la dicotomía: el **espectro** arquitectónico
3. Justificar la división (y el anti-patrón a evitar)
4. Fundamentos de estrategia de migración
5. El panorama de patrones del curso

1. Monolitos vs. microservicios

¿Qué es un monolito?

El término se usa para **dos ideas distintas**:

- **Unidad de despliegue:** toda la app se compila y despliega como un solo artefacto (WAR, binario, imagen)
- **Unidad de organización del código:** todo el código en un mismo proyecto

Suelen ir juntas, pero **no son lo mismo**.

En el curso, "monolito" = **unidad de despliegue única** — porque es el despliegue lo que los microservicios cambian.

El monolito NO es intrínsecamente malo

Ventaja	Por qué
Simplicidad operativa	Un solo proceso que desplegar y monitorizar
Transacciones ACID	Una sola BD: consistencia "gratis"
Refactorización fácil	El IDE ve todo el código
Latencia mínima	Llamadas internas = invocación de método, no red
Depuración sencilla	Un stack trace cuenta toda la historia

...pero sufre con la escala

- **Acoplamiento creciente** → *big ball of mud* (gran bola de barro)
- **Despliegues acoplados:** un cambio pequeño obliga a redesplegar todo
- **Escalado grueso:** escalar todo aunque solo un módulo lo necesite
- **Cuellos de botella organizativos:** muchos equipos, un código, "trenes de release"
- **Atadura tecnológica:** una sola pila para todos los problemas

¿Qué es un microservicio?

Servicio **desplegable de forma independiente**, modelado en torno a un **dominio de negocio**, que oculta sus datos tras una interfaz.

1. **Despliegue independiente** ← *la propiedad definitoria*
2. **Modelado por negocio** (pedidos, facturación), no por capas
3. **Propiedad de sus datos** (nadie toca su BD)
4. **Autonomía del equipo**

"Micro" no es tamaño en líneas de código

Es superficie de acoplamiento

La pregunta correcta no es "*¿cuántas líneas?*"
sino "*¿puedo desplegarlo sin pedir permiso?*"

Lo que se gana y lo que se paga

Se gana	Se paga
Despliegue independiente, releases frecuentes	Sistema distribuido: latencia, fallos parciales
Escalado selectivo (pagas solo lo que escala)	Operación mucho más compleja
Autonomía de equipos	Contratos y versionado entre servicios
Aislamiento de fallos (potencial)	Consistencia sin transacciones globales
Libertad tecnológica	Coste de plataforma (CI/CD, gateway...)

Se cambia complejidad local por complejidad distribuida.

Las falacias de la computación distribuida

Peter Deutsch, Sun Microsystems

Lo que deja de ser cierto al pasar por la red:

- La red **no** es fiable
- La latencia **no** es cero
- El ancho de banda **no** es infinito
- La red **no** es segura
- La topología **cambia...**

2. Más allá de la dicotomía

El espectro arquitectónico

No es binario: es un espectro

```
Big ball of mud
  → Monolito por capas
    → Monolito modular
      → SOA / servicios gruesos
        → Microservicios
```

No hay que elegir entre "monolito caótico" y "200 microservicios".

Los peldaños (1/2)

Big ball of mud — sin arquitectura discernible; no hay costuras por donde cortar.

Monolito por capas — hay orden, pero *técnico* (presentación / lógica / datos). Un cambio de negocio cruza todas las capas; las capas no dan fronteras extraíbles.

Monolito modular — corte *vertical* por dominios; cada módulo con su interfaz. Costuras sí extraíbles
→ peldaño previo natural a microservicios.

Los peldaños (2/2)

SOA / servicios gruesos — pocos servicios grandes, ESB central, modelo canónico corporativo, releases coordinadas. *Lección: distribuir sin descentralizar = cuellos de botella del monolito + red.*

Microservicios — grano fino por dominio, *smart endpoints / dumb pipes*, modelo por contexto, BD por servicio, autonomía de equipo. "SOA de grano fino y descentralizada".

El monolito modular

Una sola unidad de despliegue, pero módulos con **límites explícitos**:

- Interfaz pública pequeña, interior oculto
- Comunicación solo vía interfaces (idealmente verificado por herramientas)
- Esquema de datos propio por módulo (aunque compartan instancia)

Es a la vez: un destino válido (80% del beneficio sin el coste de distribuir) **y** el mejor punto de partida para migrar.

Si no eres capaz de construir un **monolito modular** bien estructurado, los microservicios **no** lo van a arreglar:
van a **distribuir el desorden** y añadirle **red**.

¿Dónde situarse en el espectro?

- **Tamaño y nº de equipos** ← el factor más determinante
- **Escalado diferencial** entre partes
- **Madurez operativa** (CI/CD, observabilidad) ← marca el límite *alcanzable*
- **Velocidad de cambio** y dónde se concentra

Y recordar:

1. La posición **no es permanente** (avanzar = migrar; retroceder = corregir)
2. **No tiene que ser uniforme** (híbridos deliberados son destino válido)
3. El movimiento tiene **orden natural** (pasar por el modular antes)

Ejercicio 1.1 + 1.2

¿Monolito culpable o inocente?

Detectar el monolito distribuido

(ver cuaderno de ejercicios)

3. Justificar la división

Empezar por el porqué

"Migramos porque queremos **X**, lo mediremos con **Y**, y sabremos que avanzamos cuando **Z**."

Objetivo	Métrica posible
Acelerar la entrega	Frecuencia de despliegue, lead time
Escalar la organización	Nº equipos autónomos
Escalar una parte	Coste infra, latencia bajo carga
Aislar fallos	Radio de impacto, MTTR

Cuándo NO merece la pena migrar

- **Equipo pequeño** (< 10–15): el coste operativo se come la ganancia
- **Dominio poco entendido**: los límites cambiarán (caro moverlos entre servicios)
- **El problema real es otro**: falta de tests, despliegues manuales → distribuir lo amplifica
- **Madurez operativa insuficiente**
- **Producto de vida corta**

Alternativas antes de dividir: re-platforming · modularización · extraer 1–2 servicios y parar

El anti-patrón: el monolito distribuido

Lo **peor** posible: muchos servicios que deben **desplegarse juntos**.

Síntomas:

- Un cambio típico toca 3–4 servicios
- Despliegues coordinados ("el viernes desplegamos todos")
- Servicios que comparten BD
- Cadenas largas de llamadas síncronas

Test rápido

Si para entregar una funcionalidad media necesitas **coordinar despliegues** de varios servicios...

no tienes microservicios:

tienes un monolito con latencia de red

4. Estrategia de migración

Incremental vs. big-bang

Big-bang (reescribir todo y cambiar de golpe): casi siempre fracasa

- Objetivo móvil (el monolito sigue vivo)
- Cero valor durante meses/años
- Riesgo concentrado en un único corte

Incremental (pieza a pieza, sistema siempre vivo):

- Pasos pequeños y **reversibles**
- Valor entregado por el camino
- Lo viejo y lo nuevo **conviven durante años** ← es el plan, no un fallo

Gestión del riesgo

El "cómo" del enfoque incremental: prácticas para que cada paso sea seguro y reversible.

- **Empieza por algo que enseñe mucho y arriesgue poco** (no el corazón crítico)
- **Separa despliegue de activación** (feature flags, enrutado gradual)
- **Mide antes y después** (sin línea base no hay prueba de mejora)
- **Plan de retirada del legacy:** la migración termina cuando el código viejo **se borra**

Secuenciar: la matriz valor x facilidad

Si migras de forma incremental, el siguiente paso es decidir el orden: qué pieza extraer primero, cuál después y cuál no tocar.

		Facilidad de extraer	
		baja	alta
Valor	alto	(2) INVERTIR después	(1) EMPEZAR AQUÍ victorias rápidas
	bajo	(4) ¿JAMÁS? dejar en monolito	(3) RELLENO no es progreso

Criterios: acoplamiento · valor · volatilidad · riesgo · datos

Buenas prácticas generales

- Trata la migración como **producto**, no como proyecto big-bang
- **No congeles el monolito** salvo en zonas de extracción activa
- Invierte pronto en la **plataforma** (el 2º servicio debe ser mucho más barato que el 1º)
- Comunica el **estado de la convivencia** (qué está en servicios, qué en el monolito, qué a medias)

Ejercicio 1.3 + 1.4

Objetivos y criterios de éxito

Secuenciar la migración

(ver cuaderno de ejercicios)

5. El panorama de patrones

El mapa del curso

Migración (Sesión 2) — *¿cómo muevo funcionalidad?*

Strangler · ACL · Branch by Abstraction · Parallel Run · Bubble Context

Descomposición (Sesión 3) — *¿dónde corto? ¿y los datos?*

DDD · bounded contexts · BD por servicio · Saga · Outbox

Operación (Sesión 4) — *¿cómo hago robusto el resultado?*

Sync/async · API Gateway · Circuit breaker · Observabilidad · Conway

Resumen de la sesión

- Monolito y microservicios son **puntos de un espectro**; el modular es destino válido
- Los microservicios compran **autonomía** pagando **complejidad distribuida**
- Antes de migrar: **objetivo, métricas con línea base, criterio de parada**
- Evita el **monolito distribuido** (test: ¿cuántos servicios toca un cambio?)
- Migración sana = **incremental**, reversible, con valor por el camino y **borrado** del legacy

¿Preguntas?

Próxima sesión: Patrones de migración

Strangler Fig · ACL · Branch by Abstraction · Parallel Run · Bubble Context